GPU Systems Engineering: A Pareto-Oriented Crash Course and the PBG Case Study

From CUDA fundamentals to the HU NLHE H100/RTX 2080 Ti case study

August 2026

Contents

Who this is for . . . 2

Part I — A GPU and CUDA systems crash course **2**

1. The big picture: a GPU is a throughput machine inside a larger system . . . 2
2. CUDA's execution model, from function call to active warps . . . 4
3. The memory hierarchy: where performance is usually won or lost . . . 6
4. A quantitative performance model . . . 8
5. The high-ROI optimization ladder . . . 10
6. Batching, vectorization, fusion, and shape design . . . 12
7. Numerical formats and mixed precision . . . 14
8. Asynchrony: streams, events, graphs, and accidental barriers . . . 15
9. Memory engineering as system design . . . 17
10. The software stack: where CUDA performance decisions are made . . . 18
11. Measurement and profiling: how to learn what the machine is doing . . . 20
12. Correctness, determinism, and portability . . . 21
13. Multi-GPU systems in one page . . . 22
14. A worked diagnosis: why a tiny transformer disappoints on a huge GPU . . . 22
15. A compact Pareto checklist . . . 23

Part II — How this project uses those ideas **24**

16. The computational problem in this repository . . . 24
17. Where the high-ROI ideas appear in the implementation . . . 27
18. H100 versus RTX 2080 Ti: not the same GPU at different speed . . . 30
19. What the H100 debugging history actually taught us . . . 31
20. What the August H100 experiment measured . . . 33
21. A Pareto-ranked experiment and optimization backlog . . . 35

22. A concrete experiment matrix . . . 37
23. Closing perspective . . . 38
Glossary . . . 38
References and repository evidence . . . 39

Status: explanatory background and engineering history, not a mathematical or runtime contract. The authoritative project contracts remain the source code, `AGENTS.md`, `CONTRACTS.md`, and the focused documents under `docs/queued/`. Hardware peak numbers are specification ceilings, not performance predictions.

Who this is for

This chapter assumes mature mathematical and computer-science instincts but no prior systems, GPU, or CUDA experience. Familiarity with asymptotic analysis, arrays, probability, and neural networks is helpful. No knowledge of computer architecture beyond "a CPU executes instructions and memory stores bytes" is required.

The first part is deliberately generic. It develops a mental model for GPU systems and a method for finding high-return optimizations without getting lost in low-level cleverness. The final part applies that model to this repository's NLHE learning pipeline, its RTX 2080 Ti and H100 environments, the failures the H100 work exposed, the fixes that followed, and the experiments that remain worth considering.

Two uses of the letter R would otherwise collide. In the generic and project hardware discussion, K denotes a number of independent model replicas. In the poker configuration, R denotes the number of retained card ranks.

Part I — A GPU and CUDA systems crash course

1. The big picture: a GPU is a throughput machine inside a larger system

The shortest useful description of a modern accelerator is:

> A GPU is a large collection of relatively simple arithmetic lanes, fed by a deep memory hierarchy and controlled by a host CPU. It is exceptionally fast when the program exposes enough similar, independent work and exceptionally easy to starve when it does not.

A conventional CPU spends substantial silicon and power making a few threads fast. Its cores have sophisticated branch prediction, large caches, speculative and out-of-order execution, and strong single-thread latency. A GPU spends more of its budget on arithmetic throughput and hardware support for keeping many threads in flight. It tolerates the latency of one memory request by running other ready work rather than making one thread complete as quickly as possible.

That difference changes the optimization objective:

- CPU question: "How quickly can this dependency chain finish?"
- GPU question: "Can thousands of arithmetic lanes keep receiving useful work?"

This is a tendency, not a law. CPUs can exploit vector and thread parallelism; GPUs can execute dependency-heavy code. The point is economic: each device is best when the program matches what the hardware bought with its transistor and power budget.

1.1 Latency, throughput, and utilization are different quantities

Suppose one request takes 10 ms on a CPU and 15 ms on a GPU. If the CPU handles one request at a time while the GPU handles 1,000 together, the GPU has worse single-request latency but vastly better aggregate throughput. Conversely, a GPU capable of an enormous peak operation rate may show poor utilization if it receives one tiny request every millisecond.

Three metrics should therefore never be conflated:

1. **Latency:** elapsed time for one unit of work.
2. **Throughput:** useful units completed per second.
3. **Utilization:** how busy some hardware resource appears.

High utilization is not automatically high useful throughput. A GPU can be busy performing padding, recomputation, copies, or a mathematically wrong objective. Low utilization is not automatically a defect either: a memory-bound kernel can saturate memory bandwidth while most arithmetic units are idle. The real objective must be stated in domain units such as examples/s, tokens/s, simulated decisions/s, or time-to-quality.

1.2 The whole machine matters

CUDA code runs inside a heterogeneous system:

```
storage -> CPU process -> host memory -> driver/runtime -> GPU memory
        -> GPU kernels -> synchronization -> results/checkpoints
```

The GPU cannot compensate for an input pipeline that stalls, a Python loop that submits work too slowly, repeated host/device transfers, an allocator that fragments memory, or a distributed collective waiting on another rank. "GPU optimization" is consequently system optimization. The unit of analysis is an end-to-end critical path, not an isolated multiplication.

1.3 The Pareto principle for accelerator work

In practice, a small set of choices often controls most performance:

- keep the working set on the accelerator;
- expose enough parallel work through batching or another honest independent axis;
- use optimized library kernels for dense linear algebra and attention;
- choose a supported low-precision format while keeping sensitive state in appropriate precision;
- eliminate unintended synchronization and excessive tiny launches;
- bound memory by actual lifetimes rather than by the sum of all possible objects; and
- measure the real workload before writing custom kernels.

The hard part is not memorizing optimization tricks. It is identifying which of these constraints is active in the current program. The Pareto approach is to buy the largest credible end-to-end gain per unit of implementation risk.

One useful decision score is informal but disciplined:

$$\text{ROI score} \approx \frac{\text{possible end-to-end gain} \times \text{confidence}}{\text{engineering cost} \times \text{correctness risk} \times \text{maintenance cost}}.$$

No one should pretend that the factors are known precisely. The formula is a reminder that a 50% microkernel opportunity covering 2% of runtime is worth at most about 1% end to end, while a mundane batching change covering 60% of runtime can dominate months of kernel work.

1.4 A running example: one million independent rows

To make the abstractions concrete, imagine that we have one million rows. Each row contains a state vector $x_i \in \mathbb{R}^{128}$, and we want to compute

$$y_i = \mathrm{ReLU}(x_i W + b), \qquad W \in \mathbb{R}^{128\times 128}.$$

The rows are independent and share W. Several implementations are possible:

1. A Python loop calls the model one row at a time.
2. A CPU thread pool assigns row ranges to cores.
3. The GPU receives one row at a time.
4. The GPU receives a large batch and calls one matrix multiplication.

All four compute the same mathematical map. They present radically different physical work. Option 3 pays host dispatch and kernel startup repeatedly while giving each launch almost no parallel slack. Option 4 turns the dominant work into a large GEMM, reuses W, and amortizes the launch. This is the basic reason "put it on the GPU" is incomplete advice: the presentation of the work usually matters more than the existence of a CUDA device.

Now change the problem so every row has its own matrix W_i. The shared-weight GEMM is no longer valid. We need batched/grouped GEMM or another explicit independent-model strategy. The fastest-looking transformation can be mathematically wrong if it accidentally shares weights. Throughout this chapter, "honest parallel axis" means an axis that exposes concurrency while preserving exactly this kind of semantic distinction.

2. CUDA's execution model, from function call to active warps

CUDA is both a programming model and a software platform. The CPU is the **host**; an NVIDIA GPU is the **device**. Host code allocates memory, prepares arguments, launches device functions called **kernels**, and coordinates dependencies. A kernel launch conceptually creates a grid of lightweight threads.

2.1 Grid, block, thread

CUDA threads are organized hierarchically:

```
kernel launch
  grid
    thread block 0
      thread 0, thread 1, ...
    thread block 1
      thread 0, thread 1, ...
    ...
```

Threads in a block can cooperate through fast on-chip shared memory and block-level barriers. Blocks should be independent enough that the hardware may run them in any order on available **streaming multiprocessors** (SMs). This independence is what lets the same kernel scale across GPUs with different numbers of SMs.

A vector addition illustrates the indexing model:

```
__global__ void add(const float* x, const float* y, float* z, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) z[i] = x[i] + y[i];
}
```

One launch creates enough blocks to cover n elements. Each logical thread computes its global index and handles one element. Real kernels often make each thread do more work, tile data cooperatively, or use multiple dimensions, but the hierarchy remains.

2.2 Warps and SIMT

The hardware schedules threads in groups of 32 called **warps**. CUDA uses the term SIMT: single instruction, multiple threads. Threads retain their own register state and may follow different logical paths, but a warp is most efficient when its active threads execute the same instruction on adjacent data.

If half a warp takes one branch and half takes another, the hardware generally executes the necessary paths with different active masks. This **divergence** does not make the result wrong, but it can reduce useful work per issued instruction. Divergence between different warps is unremarkable; divergence within a warp is the concern.

This suggests a general design pattern: group similar work together. Sort or bucket rows by shape, phase, or operation type before a large kernel when the cost of grouping is smaller than the waste it removes. Do not sort merely for aesthetic regularity; measure the trade.

2.3 SMs, resident blocks, and latency hiding

An SM owns execution pipelines, warp schedulers, a register file, caches, and shared memory. Multiple blocks and warps may be resident on an SM. When one warp waits for data or an instruction dependency, a scheduler can issue a ready warp. This is **latency hiding**.

The fraction of the hardware's theoretical resident-warp capacity actually occupied is called **occupancy**. Occupancy is limited by resources including:

- threads per block;
- registers per thread;
- shared memory per block; and
- architectural block and warp limits.

Higher occupancy can provide more latency-hiding opportunities, but 100% occupancy is not the objective. A kernel using more registers may do less spilling and finish sooner at lower occupancy. A compute-dense matrix kernel can perform well without maximal residency. Occupancy is a diagnostic constraint, not a score.

2.4 Ordinary cores and Tensor Cores

Modern NVIDIA GPUs contain general arithmetic pipelines and specialized **Tensor Cores** for matrix multiply-accumulate operations on supported shapes and data types. Neural networks spend much of their time in matrix products, so Tensor Cores can provide much higher throughput than scalar FP32 pipelines.

They are not magic neural-network cores. A framework or kernel must present compatible operations, layouts, dimensions, alignments, and data types. Small, awkward, or fragmented matrix products may realize only a small fraction of the published peak. “The GPU has 989 dense BF16 Tensor TFLOP/s” and “this model can perform 989 trillion useful operations per second” are radically different claims.

2.5 Compute capability is an executable contract

NVIDIA labels architectural feature sets with a **compute capability**. Turing devices such as the RTX 2080 Ti use 7.5; H100 uses 9.0. This affects available instructions, supported numeric formats, shared-memory limits, and which compiled device binaries can execute.

Compute capability is not the installed CUDA toolkit version. A machine can have a new toolkit and an old GPU. Native CUDA extensions should be compiled for their target architectures, and build caches should not silently reuse an SM75 artifact for an SM90 experiment. PTX just-in-time compilation can provide some forward compatibility, but it does not eliminate the need to verify the actual artifact and dispatch path.

2.6 Worked launch geometry

Return to a simple elementwise kernel over $n = 1{,}000{,}000$ values. With 256 threads per block, the grid needs

$$\left\lceil \frac{1{,}000{,}000}{256} \right\rceil = 3{,}907$$

blocks. The last block contains inactive threads beyond the array boundary; the `if (i < n)` guard makes them harmless. The runtime distributes blocks to SMs as resources become available. The source does not hard-code whether 68 or 132 SMs exist, which is how the same grid scales across the RTX 2080 Ti and H100.

Why not launch one block of one million threads? A block has an architectural thread limit and must reside on one SM. Thousands of independent blocks let the hardware load-balance across every SM. Why not use one thread per block? That would create enormous scheduling overhead and give each warp only one useful lane. Blocks commonly use a multiple of 32 threads so warps are full, but the best block size also depends on registers, shared memory, and work per thread.

Suppose a kernel uses so many registers that only two blocks can reside on an SM. Reducing block size might permit more blocks and warps, but could also make each block less efficient. The practical method is to inspect compiler resource usage, use an occupancy calculator as a constraint check, and benchmark a small set of sensible launch configurations. "256 threads" is a common starting point, not a universal constant.

3. The memory hierarchy: where performance is usually won or lost

Arithmetic is useful only when operands arrive. A simplified device hierarchy is:

```
registers (private to a thread; fastest and smallest)
    ↓
shared memory / L1 cache (on an SM)
    ↓
L2 cache (shared across SMs)
    ↓
device DRAM: GDDR6 or HBM (large, high bandwidth, high latency)
    ↓
host-device link: PCIe or a platform interconnect
    ↓
host DRAM and storage
```

The arrows should not be interpreted as a mandatory access sequence; caches and special paths complicate the machine. The important lesson is that distance and capacity increase together, while latency and the energy cost of moving a byte generally increase as well.

3.1 Bandwidth, latency, and capacity

- **Capacity** asks whether the working set fits.
- **Bandwidth** asks how many bytes per second can move under sustained load.
- **Latency** asks how long one dependency takes.

An algorithm can fit in memory yet be bandwidth-bound. It can use little bandwidth yet stall on dependent random loads. Adding HBM capacity solves an out-of-memory problem but does not automatically improve a launch-bound tiny model. These axes must be diagnosed separately.

3.2 Coalescing and structure of arrays

When threads in a warp access adjacent addresses, the hardware can combine the requests into relatively few memory transactions. Scattered or strided access can fetch many bytes that no thread uses. This property is called **coalescing**.

Consider 32 poker states. An array-of-structures layout stores all fields for state 0, then all fields for state 1:

```
[state0: actor, street, pot, ...][state1: actor, street, pot, ...]...
```

If a kernel needs only `actor`, consecutive threads jump by the full structure width. A structure-of-arrays layout stores one dense array per field:

```
actor:  [a0, a1, a2, ...]
street: [s0, s1, s2, ...]
pot:    [p0, p1, p2, ...]
```

Now adjacent threads read adjacent actors. SoA also permits fields to have different types and lifetimes. It is not universally superior—one thread using every field of one small object may prefer AoS or a hybrid—but it is a powerful default for batched GPU pipelines.

3.3 Reuse and arithmetic intensity

Define arithmetic intensity as

$$I = \frac{\text{arithmetic operations}}{\text{bytes transferred from the relevant memory level}}.$$

A matrix multiplication reuses tiles of its inputs many times, so its intensity can be high. An elementwise addition reads two values and writes one for one addition, so its intensity is low. Fusion can raise intensity by keeping an intermediate in registers or shared memory instead of writing it to DRAM and reading it back.

The qualifier "relevant memory level" matters. A kernel may be HBM-efficient but limited by L2 traffic, shared-memory bandwidth, or register dependencies. The basic roofline model introduced later usually starts with device DRAM and then becomes more detailed when evidence requires it.

3.4 Allocated, reserved, live, and peak memory

Framework memory reports are easy to misread:

- **Live/allocated tensor memory:** bytes currently owned by live tensors.
- **Reserved memory:** blocks retained by a caching allocator for reuse.
- **Peak allocated/reserved:** high-water marks since counters were reset.
- **Workspace:** temporary library or kernel storage, sometimes hidden behind an operator boundary.
- **Fragmentation/non-releasable memory:** free space inside reserved blocks that cannot satisfy the next request or be returned conveniently.

Reserved memory remaining after a step is not by itself a leak. Monotonic growth across equivalent steady-state iterations is more suspicious. A first iteration may also understate the eventual peak if later iterations create a reference cache, compiled graph pool, optimizer state, or new shape class.

3.5 Lifetime analysis beats "sum every object" budgeting

Suppose rollout needs 20 GB, evaluator completion needs 30 GB, and learning needs 40 GB. If the phases are serialized and earlier phase-only storage is released before the next phase, the lower bound is near the largest compatible phase envelope, not $20 + 30 + 40 = 90$ GB. On the other hand, a tensor retained for backward or checkpoint rollback remains live even if the source-level variable name goes out of scope.

A useful memory design starts with a lifetime table:

Object	Created	Last consumer	Replicated?	Size law
model parameters	session start	session end	K	$O(KN)$
rollout state	rollout	sealing	K	workload-dependent
completion workspace	completion	completion publication	often K	samples-dependent
activations	learner forward	backward	active cohort	$O(KBNL)$
optimizer moments	first update	session end	K	$O(KN)$

The last-consumer column is more useful than the creation site. It determines when memory can actually be reused.

3.6 Worked memory examples

Consider a warp whose 32 threads each load one adjacent FP32 value. The warp requests 128 useful bytes. On devices of compute capability 6.0 or newer, those loads can be serviced by four aligned 32-byte transactions. If instead thread t loads element $32t$, the warp touches 32 widely separated regions and may cause roughly one transaction per useful four-byte value. The arithmetic is identical; the second layout moves far more memory.

Now consider the elementwise operation

$$y_i = ax_i + y_i.$$

Ignoring cache reuse, it performs two floating operations and moves about 12 bytes per FP32 element: read x_i, read y_i, write y_i. Its arithmetic intensity is only

$$I \approx \frac{2}{12} = 0.167 \text{ FLOP/byte}.$$

At the H100 SXM specification bandwidth of 3.35 TB/s, even perfect streaming would give a device-memory roof around 0.56 TFLOP/s for this operation—far below the GPU's FP32 or Tensor Core peak. It is not a bad kernel merely because arithmetic utilization is low; it is a memory operation.

Finally, suppose three consecutive pointwise kernels each read and write a one-gigabyte tensor. In the worst simple accounting they cause several gigabytes of avoidable round trips. Fusing them can be worth more than tuning the arithmetic instructions because the saved bytes dominate. This is why a profiler needs both kernel time and traffic, not only FLOP/s.

4. A quantitative performance model

Optimization is much easier when "slow" is decomposed. For an end-to-end iteration, write schematically

$$T_{\text{total}} = T_{\text{host}} + T_{\text{launch}} + T_{\text{compute}} + T_{\text{memory}} + T_{\text{transfer}} + T_{\text{sync}} + T_{\text{communication}} + T_{\text{I/O}},$$

with the warning that terms can overlap and therefore do not always add as independent wall times. A trace should reveal the critical path and the amount of overlap.

4.1 Amdahl's law: optimize coverage, not glamour

If fraction p of runtime is improved by factor s, the ideal total speedup is

$$S = \frac{1}{(1-p) + p/s}.$$

Even making 5% of runtime infinitely fast yields only $1/(0.95) \approx 1.053\times$. Making a 60% component twice as fast yields $1/(0.4+0.3) \approx 1.43\times$. Before implementing a custom kernel, measure its coverage and calculate its maximum possible end-to-end value.

Amdahl's law describes fixed-work or strong-scaling intuition. Gustafson's law captures another important possibility: a larger accelerator may let us solve a proportionally larger problem in similar time. In simulation and learning, that can be the more valuable use—more independent trajectories, hands, or experiments—provided the extra work is scientifically useful.

4.2 The roofline model

Let $P_{\max}$ be peak arithmetic throughput and $B_{\max}$ sustained memory bandwidth. For arithmetic intensity I, a simple upper bound is

$$P_{\text{attainable}} \leq \min(P_{\max}, IB_{\max}).$$

At low intensity, the sloped $IB_{\max}$ roof indicates memory pressure. At high intensity, the flat $P_{\max}$ roof indicates compute pressure. The intersection

$$I^* = P_{\max}/B_{\max}$$

is the ridge point.

For the H100 SXM dense BF16 specification, the rough HBM ridge point is

$$I^* \approx \frac{989 \text{ TFLOP/s}}{3.35 \text{ TB/s}} \approx 295 \text{ FLOP/byte}.$$

This uses two specification ceilings and is therefore only a first-order guide, but it conveys scale. The preceding SAXPY-like operation at 0.167 FLOP/byte lies overwhelmingly on the bandwidth side. A large tiled matrix multiplication may reuse inputs enough to exceed hundreds of FLOPs per HBM byte and approach the compute side. A small or skinny GEMM may have adequate algorithmic intensity but still miss peak because its shape cannot fill Tensor Core tiles or all SMs.

This model is valuable because it prevents category mistakes. If a kernel moves 12 bytes for one FP32 addition, buying more Tensor Core throughput cannot help much. If a large matrix multiplication is compute-bound, shaving a small metadata read is unlikely to matter. Real machines have several roofs—L1, L2, HBM, scalar pipelines, Tensor Cores, instruction issue—and achieved ceilings depend on shape, but the first-order classification is already useful.

4.3 Launch-bound work

A third regime is especially important for small models: the GPU may be neither compute-bound nor bandwidth-bound because kernels are too short and the host cannot submit them without gaps. If a kernel runs for a few microseconds and Python/framework dispatch consumes a comparable interval, a timeline shows thin GPU rectangles separated by empty space.

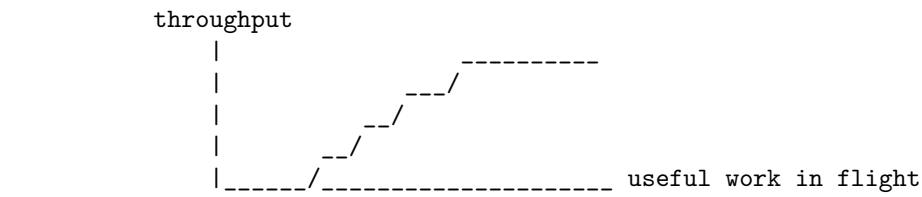
Possible responses include:

- increase useful batch size;
- fuse adjacent pointwise operations;
- compile a stable region;
- use a vectorized or `foreach` optimizer;
- capture a repeated static schedule in a CUDA Graph; or
- move a host-controlled numerical loop into one device operation.

The order matters. CUDA Graphs can replay a bad collection of tiny, redundant kernels with lower launch cost, but they do not remove the redundant work.

4.4 Parallel slack and the saturation curve

Let W be useful work submitted together. Throughput commonly follows a curve that rises quickly and then plateaus:



Below saturation, a larger batch, more roots, more sequences, or more replicas can amortize fixed overhead and fill more SMs. Near saturation, extra work adds memory pressure and latency without much throughput. Beyond capacity, it causes out-of-memory failure or forces microbatching.

The experimenter's job is to find the plateau in domain throughput, not to maximize a batch-size knob.

4.5 Queueing intuition

If work arrives at average rate λ and spends average time W in a stable subsystem, Little's law gives average in-flight work

$$L = \lambda W.$$

This is a useful systems sanity check. A pipeline seeking high throughput must keep enough work in flight to cover service latency. But queues that grow without bound indicate arrival rate above service rate; they increase memory and latency rather than throughput. Fixed-capacity queues and backpressure make this failure explicit.

5. The high-ROI optimization ladder

The following order is not absolute, but it is a strong default because each stage improves the evidence needed for the next.

5.1 Level 0: define correct useful work

Before making anything faster, define what must remain invariant:

- objective and normalization;
- sampling distribution and RNG identity requirements;
- number of optimizer transitions;
- data and model versions;
- failure atomicity and checkpoint semantics;

- accepted numerical tolerance; and
- domain work counters.

A faster biased sampler or half-sized training schedule is a different experiment, not a systems optimization. Padding may be physically executed but must not be counted as useful presentations. Recomputed validation passes must not be hidden in a throughput denominator.

The best performance regression test is often an identity such as

$$\text{presentations} = K \times \text{steps} \times \text{batch size},$$

plus exact per-phase work ledgers and model-state comparisons.

5.2 Level 1: measure end to end and by phase

Record synchronized wall time around the user-visible operation and device event time around GPU ranges. Break the iteration into phases that correspond to real ownership boundaries. Count useful work, not inferred work.

At minimum, collect:

- wall time and GPU time;
- domain throughput;
- allocated/reserved current and peak memory;
- phase durations;
- batch and shape histograms;
- host/device copy bytes;
- synchronization count;
- kernel count and visible launch gaps; and
- hardware/software identity.

Warm compilation, autotuning, lazy initialization, and allocator state before steady-state timing. Report cold-start costs separately because they are real but answer a different question.

5.3 Level 2: remove movement and synchronization mistakes

High-confidence wins include:

- keep intermediate numerical state on the device;
- batch small host/device transfers;
- avoid `.item()`, `.cpu()`, or printing a device tensor inside a hot loop;
- replace device-wide synchronization with stream dependencies or a later necessary boundary;
- use contiguous/coalesced layouts for dominant access patterns; and
- allocate stable arenas or reuse buffers rather than repeatedly allocating variable temporary objects.

Some host reads are necessary for orchestration, error handling, or reports. The goal is to place them at explicit boundaries rather than let innocent-looking framework calls serialize every step.

5.4 Level 3: expose more useful parallel work

This is often the largest win. Candidate axes include examples, tokens, trajectories, simulation roots, parameter replicas, environments, or independent requests. Choose an axis whose aggregation does not change the mathematics.

There is a crucial difference between **logical batch size** and **physical microbatch size**. A logical batch defines the statistical update. Physical microbatches are an execution decomposition used to fit memory. If a logical batch of 256 is split into physical slices, loss weights must reproduce the same update; otherwise memory tuning has silently changed the objective.

5.5 Level 4: use the best existing primitives

Prefer mature libraries for GEMM, batched GEMM, reductions, convolution, and scaled dot-product attention. They encode years of architecture-specific tuning. Verify the actual dispatch rather than assuming an API name guarantees a particular kernel.

For example, PyTorch scaled dot-product attention can select Flash, memory-efficient, or math implementations depending on dtype, shape, mask, layout, device, and version. Configuring `sdpa` establishes the requested operator, not proof that every call used Flash attention.

5.6 Level 5: reduce framework and launch overhead

Once work and shapes are stable:

- use `torch.compile` on bounded hot regions;
- inspect graph breaks and recompilations;
- use `foreach` or fused optimizers where semantics match;
- capture repeated static schedules in CUDA Graphs; and
- cache only a bounded number of shape-specialized variants.

Compilation can fuse elementwise work and reduce Python overhead. CUDA Graphs move repeated launch construction out of the replay path. Both can increase memory use through cached code, workspaces, and private pools. Dynamic shapes, data-dependent control flow, allocations, and unsupported operations can limit capture. Treat "compiled" and "graphed" as measured variants, not upgrades by definition.

5.7 Level 6: write a custom kernel only for a proven target

A custom CUDA, Triton, or CUTLASS kernel is justified when profiling identifies a dominant operation with a stable contract that existing primitives execute poorly. Good targets often combine several of:

- repeated reads/writes of short-lived intermediates;
- host-controlled loops over many small rows;
- irregular but bounded reductions;
- an operation whose semantic structure permits a single owning block; or
- a major matrix shape that generic dispatch consistently mishandles.

The kernel needs a correctness oracle, adversarial boundary tests, dtype and architecture coverage, resource/occupancy analysis, and an end-to-end retention threshold. A microbenchmark win is necessary evidence, not sufficient evidence.

6. Batching, vectorization, fusion, and shape design

These words are often used interchangeably even though they solve different problems.

6.1 Batching shared-weight work

Given one weight matrix $W \in \mathbb{R}^{D \times E}$ and a batch $X \in \mathbb{R}^{B \times D}$, ordinary batching computes

$$Y = XW.$$

Increasing B produces a larger GEMM and amortizes model-call overhead. The examples share weights. This is the normal data-batch axis.

6.2 Batching independent replicas

For K independent models,

$$X \in \mathbb{R}^{K \times B \times D}, \qquad W \in \mathbb{R}^{K \times D \times E},$$

and

$$Y_k = X_k W_k.$$

This is a batch of independent GEMMs. Flattening K into the data batch and using one shared W would be mathematically wrong. Valid implementations include batched matrix multiplication, grouped GEMM, a functional vectorization such as `vmap`, or concurrent independent stream work. Parameters, gradients, optimizer moments, clipping norms, RNG, and failure state remain replica-local.

Replica batching is attractive for tiny models because it exposes more work without increasing one experiment's statistical batch. It is also expensive: persistent model and optimizer state scales with K, while activations scale with the active cohort KB. The right K is a measured throughput/memory plateau, not "as many as fit."

6.3 Padding, bucketing, and grouped operations

Variable shapes force a choice:

- **Padding:** regular kernels, wasted arithmetic and memory.
- **Bucketing:** several regular shape classes, grouping overhead and tails.
- **Grouped/ragged kernels:** less padding, more complex dispatch and possibly lower kernel efficiency.

Measure both physical and useful work:

$$\text{padding efficiency} = \frac{\text{valid elements or tokens}}{\text{physically processed elements or tokens}}.$$

A padded kernel can win even with lower padding efficiency because its GEMMs are much better. A grouped kernel earns its complexity only if tail waste is a material fraction of end-to-end time.

6.4 Operator fusion

Suppose three pointwise operations each read and write an N-element tensor. Separate kernels may make several DRAM round trips and incur three launches. A fused kernel can load once, compute in registers, and store once. That reduces both memory traffic and launch count.

Fusion is less obviously helpful when it:

- creates register pressure and spills;
- combines stages with incompatible parallel structure;
- prevents a highly optimized library call;
- lengthens live ranges and increases memory; or
- explodes the number of compiled shape variants.

Fuse data movement and cheap elementwise work around dominant operators first. Do not attempt to replace cuBLAS or a fused attention implementation without strong shape-specific evidence.

6.5 Model shape is a systems parameter

Two models with equal parameter count can have very different efficiency. Tensor Cores prefer certain dimension multiples and sufficiently large matrix tiles. Many narrow projections may launch more kernels and achieve less reuse than fewer well-shaped ones. Attention cost grows approximately quadratically with sequence length for dense attention:

$$\text{score work} = O(BHN^2D_h),$$

while projection work is closer to $O(BND^2)$. A minority of long sequences can dominate runtime.

Changing the model solely for speed is a scientific change. It should be factorized from an execution comparison: first compare execution strategies on the same model, then evaluate a smaller or differently shaped architecture with its own quality gate.

7. Numerical formats and mixed precision

A floating-point format trades exponent range, significand precision, storage, and arithmetic throughput.

Format	Typical storage	Main intuition
FP64	8 bytes	large precision/range; expensive on many GPUs
FP32	4 bytes	conventional training/control precision
TF32	internal Tensor Core mode	FP32 range with reduced multiply precision
FP16	2 bytes	more significand bits than BF16, much smaller exponent range
BF16	2 bytes	FP32-like exponent range, fewer significand bits
FP8	1 byte	very high throughput, requires careful scaling and support

Exact bit layouts and behavior are more nuanced, but one practical contrast is central: FP16 more readily overflows or underflows because of its exponent range, while BF16 preserves roughly FP32's dynamic range at lower mantissa precision. This often makes BF16 operationally convenient for training on hardware that supports it.

7.1 Mixed precision is a policy, not a dtype switch

A robust training policy may use:

- BF16/FP16 inputs and activations for Tensor Core operations;
- FP32 accumulation where the library supports it;
- FP32 loss reductions, norms, or numerically sensitive probabilities;
- FP32 master parameters and optimizer moments; and
- explicit conversion only at declared boundaries.

Autocast chooses dtypes for supported operations; it does not decide the scientific tolerances or storage ownership of the application. A tensor written into a preallocated BF16 arena must be intentionally converted at that semantic boundary. Letting an arbitrary dtype mismatch cast silently can hide a bug in an integer address, mask, or ownership field.

7.2 Numerical equivalence cannot mean bit identity

Floating-point addition is not associative:

$$(a + b) + c \neq a + (b + c)$$

in general. Parallel reductions, fused instructions, Tensor Cores, and compiler reordering can alter roundoff. Correctness gates should combine:

- exact comparisons for integer state, addresses, masks, samples, and RNG;
- tolerance-based comparisons for outputs and losses;
- relative L_2 error and cosine similarity for large gradients;
- finite-value and range invariants;
- one-step optimizer-state comparison; and
- longer replay drift bounds.

Thresholds must be fixed before selecting a faster candidate. Loosening them after observing a failure turns validation into rationalization.

7.3 Hardware support is non-negotiable

Turing Tensor Cores support FP16 matrix inputs but not BF16. Native BF16 Tensor Core support arrived with Ampere and is present on Hopper. A BF16 production path can therefore be tested semantically only on supporting hardware. An RTX 2080 Ti FP32 smoke can validate control flow, storage, CUDA residency, and many invariants, but it cannot validate H100 BF16 dispatch or numerical behavior.

7.4 A worked mixed-precision training step

Consider a linear layer with a million input activations. Storing those activations in FP32 requires about 4 MB; BF16 requires about 2 MB. Saving the input for backward therefore halves that particular activation buffer, and a Tensor Core matrix multiplication may become much faster. It does **not** follow that the complete training step uses half the memory. FP32 master weights, FP32 gradients, Adam's two FP32 moment vectors, allocator workspaces, and non-floating metadata may be unchanged.

A defensible step might look conceptually like this:

```
BF16 input and saved activations
  -> BF16 matrix operands with FP32 accumulation
  -> FP32 logits for a sensitive normalization
  -> FP32 scalar loss and gradient-norm reduction
  -> BF16/FP32 backward selected per operator
  -> FP32 gradient clipping and Adam state transition
```

The cast locations are part of the interface. For example, suppose a producer builds an FP32 feature row and the next owner is a declared BF16 model-input arena. Converting exactly at that publication boundary is meaningful: the address-like integer fields remain exact, and downstream code can rely on the arena's dtype. Converting the entire record indiscriminately would corrupt its type contract; refusing the declared floating conversion would make the mixed precision design unusable.

FP16 and BF16 also illustrate why “16 bit” is not a complete specification. The largest finite FP16 value is 65,504, whereas BF16 has approximately the same exponent range as FP32. BF16 consequently tolerates a much wider scale range, but its coarser significand means nearby large values can round to the same number. The right question is not which format is universally accurate; it is which operations need range, which need fine relative precision, and which error the objective and optimizer can tolerate.

8. Asynchrony: streams, events, graphs, and accidental barriers

CUDA kernel launches are normally asynchronous with respect to the host: the CPU enqueues work and continues. A **stream** is an ordered sequence of device operations. Operations within a stream execute in order; operations in different streams may overlap when dependencies and resources permit.

8.1 Events express device dependencies

A CUDA event can be recorded after one operation and waited on by another stream:

```
model stream:  [forward] --record ready-- [next forward]
commit stream:               wait ready-- [publish]
```

This allows a precise dependency without stopping the whole device or forcing the CPU to wait. Device-wide synchronization is a much larger hammer. It is appropriate at measurement, report, checkpoint, teardown, or explicit failure boundaries; it is usually suspicious inside a fine-grained hot loop.

8.2 Concurrency is not guaranteed by using many streams

Two kernels overlap only if:

- dependencies allow it;
- they are actually enqueued early enough;
- together they fit available SM/register/shared-memory resources;
- no implicit synchronization intervenes; and
- the device supports the needed concurrency.

A large kernel may already occupy the machine. Adding streams then changes nothing. Conversely, concurrent tiny independent kernels can fill otherwise idle resources. A timeline, not source-code stream count, establishes overlap.

8.3 CUDA Graphs

A CUDA Graph records a repeated operation/dependency schedule and instantiates it for low-overhead replay. It is well suited to stable shapes, addresses, and control flow. It can greatly reduce CPU launch work for small repeated steps.

Costs and constraints include capture warmup, static buffers, restrictions on some operations, graph-private memory pools, RNG handling, and a cache problem when many shapes require different graphs. A good design has a small explicit set of graph classes and an eager fallback. Graph replay should be compared against a warmed eager/compiled baseline with identical useful work.

8.4 The hidden synchronization checklist

In a GPU framework, these operations often deserve inspection:

- converting a device scalar to a Python number;
- copying a tensor to CPU;
- host-side branching on device data;
- exception or assertion paths that inspect device state;
- allocator phase changes and cache-emptying calls;
- distributed collectives in inconsistent order;
- default-stream work interacting with non-default streams; and
- timing without an explicit synchronization boundary.

The answer is not to ban them. It is to make them rare, intentional, and visible in a trace.

8.5 A worked two-stream pipeline

Suppose batch i needs 3 ms of model work and 1 ms of publication work. A single stream executes

```
model(i) [---] publish(i) [-] model(i+1) [---] publish(i+1) [-]
```

for a nominal 8 ms across two batches. If publication uses few enough resources and batch $i + 1$ is independent, two streams can instead express

```
model stream:   model(i) [---] model(i+1) [---]
commit stream:               publish(i) [-]   publish(i+1) [-]
```

An event recorded after `model(i)` permits `publish(i)` to start without a device-wide barrier. The first publication may overlap the second model, so the idealized makespan falls toward 7 ms. It cannot fall to 6 ms merely because two streams exist: each stream preserves its own order, publication depends on its model, and the two kernels may contend for the same resources.

Now add `loss = device_loss.item()` between the two model launches. The host must wait until the first loss is available before it can enqueue later work; the intended overlap may vanish. This is why source-level asynchrony is not enough. The proof is a timeline showing early enqueue, event dependencies, and actual overlap on the target device.

9. Memory engineering as system design

GPU memory is not just a bag into which tensors are placed. It is an ownership and scheduling constraint. A reliable accelerator system answers four questions for every material allocation:

1. Who owns it?
2. What exact shape and dtype may it have?
3. What is its last consumer?
4. What happens if the producer fails or exceeds capacity?

9.1 Static arenas versus dynamic tensor allocation

Dynamic framework allocation is convenient and often entirely adequate. For a large irregular pipeline, however, thousands of differently shaped allocations make capacity hard to prove and failure hard to make atomic. A fixed arena can instead provide typed views at aligned offsets and monotone row reservations:

```
reserve [begin, end) -> fill every declared field -> publish tail
```

Readers observe only the published prefix. If filling fails, the unpublished reservation can be discarded. This resembles a database transaction more than a conventional Python list.

Advantages include predictable addresses, exact analytical storage, fewer allocator calls, easy high-water accounting, and graph-friendly stability. Costs include internal slack, up-front capacity work, more rigid schemas, and the need for explicit lifetime discipline. Static storage is high ROI only when the workload has useful bounds and correctness benefits from the contract.

9.2 Capacity and microbatching are different dimensions

This distinction prevents a common failure. A **cumulative capacity** covers everything retained over a phase: for example, one row per root across an entire rollout. A **workspace or pack capacity** covers only the rows processed concurrently: for example, 256 roots in one model call.

If cumulative storage is accidentally sized to one pack, several packs may run successfully before deterministic overflow. If workspace is sized to the full cumulative count, the program may waste tens of gigabytes. Each bound should be named by lifetime:

$$\text{cumulative rows} \neq \text{maximum simultaneous rows}.$$

9.3 Preflight admission

A capacity compiler can transform a small workload description into all derived row counts, tensor shapes, workspaces, and phase peaks. It should reject an impossible run before expensive rollout begins.

For each phase ϕ, a conservative admission inequality is

$$M_{\text{persistent}} + M_{\text{live},\phi} + M_{\text{workspace},\phi} + M_{\text{allocator/graph},\phi} + M_{\text{reserve}} \leq M_{\text{device}}.$$

The reserve is not a substitute for missing formulas. It covers residual uncertainty, library workspaces, fragmentation, and operational safety. Compare the analytical prediction with measured peaks and refine the model using held-out configurations. Do not fit a line to one point and call it a proof.

9.4 Failure atomicity

Accelerator failure can arrive late: a nonfinite loss after forward, an OOM in backward, a distributed peer failure, or a capacity overflow after partial work. If some replicas update parameters and others do not, an independent-experiment batch is corrupted. If queue tails publish before payloads are complete, consumers see invalid state.

Transactional design separates:

- provisional work;
- validation/failure agreement;
- committed optimizer or storage state; and
- durable run/checkpoint publication.

Rollback state costs memory, so it belongs in the capacity model. A "preflight" that recomputes the full learner merely to avoid rollback may be correct yet double the dominant work; the system should measure that cost explicitly.

10. The software stack: where CUDA performance decisions are made

Application code normally sits above several layers:

```
Python model / simulation
  -> PyTorch dispatcher and autograd
    -> compiler/vectorization layers
      -> ATen operators
        -> cuBLAS, cuDNN, custom CUDA/Triton/CUTLASS kernels
          -> CUDA runtime/driver
            -> GPU
```

A single line of Python may launch one highly optimized kernel, dozens of small kernels, or a fallback implementation. Conversely, a Python loop can be cheap if each iteration launches a large saturating operation. Source-line count is not a performance model.

10.1 Eager execution

PyTorch eager mode dispatches operations as Python executes them and constructs an autograd graph for differentiable work. It is productive, observable, and a valuable correctness oracle. Its weaknesses appear when many small operations create launch gaps or write materialized intermediates that a compiler could fuse.

10.2 Autograd and activation ownership

Backward needs saved forward intermediates. The apparent forward working set therefore understates training memory. Activation checkpointing trades extra forward recomputation for lower retained activation memory. It is useful when memory is binding and recomputation is cheaper than a smaller useful batch, but harmful if launch or compute time is already the main limit.

An accidental `no_grad`, in-place mutation of a saved tensor, detached copy, or incorrect functional parameter view is a correctness error. Vectorized replica training needs gradient tests at the flat parameter level, not only matching forward outputs.

10.3 Functional transforms and `vmap`

A module normally owns one parameter set. Functionalization makes parameters explicit inputs. Stacking K parameter sets and applying `vmap` can ask the framework to vectorize a function over the replica axis while retaining one canonical model definition.

This is elegant when every operator has an efficient batching rule. A fallback may instead loop or introduce awkward layouts. Attention backward, scatter, indexing, mutation, and per-replica randomness deserve direct inspection. A successful `vmap` call establishes semantics, not efficient dispatch.

10.4 Explicit batched and grouped linear algebra

When vectorization dispatch is poor, an explicit replica-native path can store linear weights as `[K, out, in]` and use BMM/grouped GEMM. This gives direct control over strides, padding, and the folding of replica and example axes into attention batches. It also duplicates implementation surface and complicates checkpoint conversion.

The clean compromise is one scientific model/loss contract with a narrow, tested execution backend. Keep the ordinary module as a correctness oracle; retain the specialized backend only if it produces a material full-step gain.

10.5 `torch.compile`

`torch.compile` captures compatible PyTorch regions and uses a backend such as Inductor to generate optimized code. Potential benefits include pointwise fusion, lower dispatch overhead, layout changes, and CUDA Graph integration.

Watch for:

- graph breaks;
- compilation repeated for new shapes or scalar values;
- cold-start time included in steady-state benchmarks;
- numerical changes from reordering;
- larger code/workspace caches; and
- fallback operators dominating the supposedly compiled region.

Compile a bounded learner replay before an entire dynamic simulation. Stable leaf functions are much easier to reason about than data-dependent queue orchestration.

10.6 Custom extensions

A C++/CUDA extension can combine a host loop into one compiled launch sequence or implement an operation absent from PyTorch. Its build is architecture-specific state. Record toolkit, compiler, ABI, PyTorch version, target compute capabilities, source revision, and cache directory. A cache key that omits any of these can reuse an incompatible binary.

11. Measurement and profiling: how to learn what the machine is doing

There are three complementary levels of evidence.

11.1 Application metrics

Application metrics answer whether the system accomplished the intended work:

- useful simulations or decisions per second;
- useful learner presentations per second;
- optimizer transitions per second;
- time-to-quality or time-to-statistical precision;
- exact sample count and padding fraction; and
- phase time and memory.

These decide whether an optimization matters. Hardware counters explain why.

11.2 Timeline profiling

Nsight Systems or a framework trace shows CPU threads, CUDA API calls, kernels, copies, streams, events, and communication over time. It answers:

- Is the GPU idle between kernels?
- Is the host submitting work quickly enough?
- Are supposedly concurrent lanes actually overlapping?
- Where are device-wide synchronizations?
- Do H2D/D2H copies sit on the critical path?
- Are distributed ranks waiting on collectives?
- Which phase owns the long interval?

Annotate domain phases with NVTX ranges. A raw sequence of kernel names is much harder to map back to the system.

11.3 Kernel profiling

Nsight Compute examines selected kernels. Useful questions include:

- achieved versus shape-specific compute throughput;
- HBM/L2/L1 traffic and hit rates;
- occupancy and the resource limiting resident blocks;
- warp execution efficiency and divergence;
- memory-load coalescing;
- instruction mix and issue stalls; and
- whether Tensor Core instructions were actually issued.

Profilers perturb execution and can serialize kernels. Use separate unprofiled timing runs and bounded profiling captures. Do not feed profiler-perturbed wall times into the main throughput result.

11.4 `nvidia-smi` is operational telemetry

`nvidia-smi` is excellent for confirming the selected GPU, memory use, power, temperature, clocks, compute processes, ECC state, and whether another job is present. Its utilization percentage is a coarse sampling signal. It cannot prove Tensor Core efficiency, identify a memory bottleneck, or validate Flash attention dispatch.

11.5 A trustworthy benchmark protocol

A performance comparison should:

1. pin exact hardware, software, source, config, and data/replay identity;
2. run on an idle exclusive device when possible;
3. separate initialization/compile/autotune from steady state;
4. synchronize immediately around the timed boundary;
5. compare identical useful work;
6. alternate candidates to reduce drift and thermal/order bias;
7. collect enough repeats to show variability;
8. report median and dispersion, not only the best run;
9. retain raw machine-readable artifacts; and
10. verify clean device teardown.

For expensive rentals, prepare a resumable experiment matrix and every command before the rental begins. Write progress atomically after each completed unit. A timeout with partial evidence is much better than an empty log.

12. Correctness, determinism, and portability

Performance engineering changes schedules. Schedules influence RNG consumption, floating-point reduction order, allocator state, and failure timing. Correctness cannot be bolted on after the fast path exists.

12.1 Logical randomness

If a random draw depends on which queue happens to run first, changing a microbatch or stream schedule changes the experiment. A stronger design derives randomness from a logical address such as

$$(\text{seed}, \text{iteration}, \text{root}, \text{node}, \text{purpose}, \text{attempt}).$$

Then physical batching may change without changing logical samples. Explicit per-replica generators are another workable design when their consumption order is part of the contract. The key is to decide, document, and test the invariance boundary.

12.2 Determinism has levels

Useful reproducibility profiles include:

- **bitwise:** same bits on a fixed software/hardware stack;
- **seeded/numerical:** same logical samples with bounded floating drift;
- **throughput:** allows nondeterministic fast kernels while preserving statistical and state contracts.

Demanding bit identity everywhere can reject valuable optimized reductions. Demanding no identity anywhere makes debugging scientific regressions nearly impossible. Different workflows can select different profiles explicitly.

12.3 Architecture portability

A test on an older GPU can validate shared CUDA logic while missing:

- unsupported numeric formats;
- different library dispatch;
- architecture-specific instructions;
- larger-memory geometry;
- race timing exposed only at higher concurrency;
- new framework compatibility defects; and
- native-extension build errors for the new SM target.

Portability therefore uses a ladder: fast local tests on the available device, architecture-specific smoke tests, then production-scale tests on the target. "Works on CUDA" is not a single binary property.

13. Multi-GPU systems in one page

Multiple GPUs introduce a network and a distributed state machine. CUDA-aware collectives are commonly provided by NCCL:

- all-reduce combines gradients or statistics across ranks;
- broadcast distributes one rank's data;
- all-gather collects shards from every rank;
- reduce-scatter combines and partitions data; and
- point-to-point operations express explicit exchange.

For data-parallel training, a rough step model is

$$T_{\text{step}} \approx T_{\text{local compute}} + T_{\text{exposed communication}} + T_{\text{imbalance}} + T_{\text{synchronization}}.$$

Communication may overlap backward compute, but small models often have too little compute to hide collective latency. All ranks must enter collectives in compatible order. One rank failing while others wait can look like a hang.

Scale only after one GPU is efficient enough and the scientific global-batch semantics are explicit. Compare strong scaling (same total work, more GPUs) and weak scaling (more total work with GPU count) separately. Aggregate throughput can rise while per-item latency and cost efficiency worsen.

14. A worked diagnosis: why a tiny transformer disappoints on a huge GPU

Imagine a transformer whose one training step contains many small projections, normalizations, gathers, and optimizer operations. Moving it from an RTX 2080 Ti to an H100 gives far more memory bandwidth and Tensor Core peak, yet observed throughput improves only modestly.

Start with hypotheses, not code:

1. **Not enough work:** GEMMs are too small to fill H100 Tensor Cores.
2. **Launch bound:** the CPU submits hundreds of short kernels with gaps.
3. **Memory bound:** gathers and pointwise operators dominate matrix work.
4. **Bad dispatch:** attention uses a math fallback or incompatible layout.
5. **Serialization:** independent replicas run one after another.
6. **Recomputation:** a safety preflight performs forward/backward twice.
7. **Measurement error:** cold compilation or different useful work pollutes the comparison.

The experiment sequence should be:

1. Reconcile exact work and isolate learner time.
2. Capture a timeline for launch gaps and serialization.
3. Record shapes and prove attention/GEMM dispatch.
4. Increase the cheapest honest parallel axis until throughput plateaus.
5. Compile or graph a fixed replay if launch gaps remain.
6. Vectorize optimizer control if it remains visible.
7. Try replica-axis `vmap`, explicit BMM, or stream waves against the same serial replay.
8. Profile only dominant remaining kernels.
9. Write a custom kernel only if its Amdahl ceiling is worthwhile.

This ordering distinguishes "the GPU needs more work" from "each unit of work is inefficient." Both can be true, but they require different fixes.

15. A compact Pareto checklist

Before optimizing:

- What is the useful-work numerator?
- What must remain exact, and what may differ within tolerance?
- What percentage of end-to-end time can the proposal affect?
- Is the target launch-, memory-, compute-, communication-, or capacity-bound?

Before increasing batch or concurrency:

- Is the axis scientifically independent or correctly normalized?
- Which state scales with K, B, sequence length, or sample count?
- Are cumulative and simultaneous capacities separate?
- Does throughput plateau before memory fills?

Before using mixed precision:

- Does the exact GPU support it?
- Which state remains FP32?
- Where are casts allowed?
- What are the predeclared numerical gates?

Before compiling or graphing:

- Are shapes and addresses stable?
- Are graph breaks/recompilations bounded?
- Is cold-start time reported separately?
- Is private-pool/cache memory admitted?

Before writing a custom kernel:

- Is the operation dominant in a real trace?
- What is the maximum end-to-end speedup?
- Is there a simple trusted oracle?
- Can a library, fusion, layout, or batching change solve it first?
- What measured retention threshold will justify maintenance?

Part II — How this project uses those ideas

16. The computational problem in this repository

This repository trains strategies for configurable no-limit Texas Hold'em cash poker, an imperfect-information game. The active game has three per-run dials:

- players $P \in \{2, \dots, 6\}$;
- equal stack depth $S \in [8, 128]$ big blinds in 0.5 bb increments; and
- retained ranks $R \in \{4, \dots, 13\}$ with all four suits.

It requires $2P + 5 < 4R$, uses 0.5/1 small/big blinds, no ante or rake, and equal stacks. The documented maximal player-node count in one trajectory is

$$4P + (S - 1)(P - 1) = SP + 3P - S + 1.$$

Unlike training on a static image dataset, one iteration constructs stochastic game trajectories, performs policy inference at decision nodes, handles chance and terminal states, evaluates hidden-card outcomes, propagates values through a topology, creates aligned training targets, and only then performs learner updates. Shapes vary by street, public history, legal action count, hidden-hand domain, rollout mode, and estimator.

The central active pipeline is:

```
strict config and immutable capacity plan
  -> fixed accelerator-resident storage owners
  -> family-scheduled rollout
       model -> sample -> transition -> atomic publication
  -> sampled showdown or Rao--Blackwell completion
  -> compact topology and reverse layers
  -> layered target propagation
  -> aligned learner sampling
  -> forward/loss/backward and dense FP32 optimizer transition
  -> metrics and atomic checkpoint
```

CUDA supports every production workflow. MPS is limited to supported inference paths. CPU work is intentionally small orchestration such as config parsing, hashing, checkpoint I/O, topology metadata, and report formatting; it does not own production rollout tensors, model forwards, loss/backward, posterior work, or showdown evaluation.

16.1 Poker and information sets in one page

In heads-up no-limit Hold'em, each player receives two private cards. Up to five public board cards arrive over the flop, turn, and river. On each betting street a player may fold, check/call when legal, or raise to a legal amount. The hand ends by a fold or by comparing five-card poker hands at showdown. Stacks, bets, board cards, and action history are public; hole cards are not.

That hidden information changes the learning object. A policy cannot condition on the opponent's actual private cards, because the player does not observe them. It conditions on an **information set**: everything the acting player can know, together with a belief or reach distribution over compatible hidden hands. Two world states that differ only in an unseen opponent hand belong to the same public decision problem. Revealing that hand to the network during training would create a policy that cannot be executed at play time.

The project consequently carries vectors over private-hand domains. Before the flop, a full 52-card deck has 169 strategically distinct rank/suitedness classes: 13 pairs, 78 suited non-pairs, and 78 offsuit non-pairs. After public cards appear, suit-specific collisions matter, so the natural domain is the $\binom{52}{2} = 1,326$ concrete two-card combinations. The network produces policy/value information over these domains and the learner selects or weights rows according to the estimator contract.

This explains why one "poker example" is much larger than one scalar feature row. It contains public geometry, action history, and distributions over many possible private states. It also explains the strict evaluator boundary: showdown and completion are large hidden-state calculations, not incidental Python rules.

16.2 A concrete HU 32 bb shape calculation

Take the selected baseline $P = 2$, $S = 32$, $R = 13$, with two learned raise components. The repository's geometry formulas give:

Quantity	Calculation	Result
Deck cards	$4R$	52
Concrete private hands	$\binom{52}{2}$	1,326
Preflop private classes	R^2	169
Maximum player nodes	$SP + 3P - S + 1$	39
History tokens	39 actions + 3 street separators	42
Public feature width	$8 + 10P$	28
History feature width	$10 \cdot 42$	420
Retained preflop row width	$28 + 2 \cdot 169 + 420$	786
Retained postflop row width	$28 + 2 \cdot 1,326 + 420$	3,100

The maximum structural trajectory has 39 player nodes, three chance/street nodes, and one leaf: 43 total nodes before topology compaction. These are capacity bounds, not predictions of the average hand length.

The transformer sequence length is different from the retained flat feature width. Under the pilot configuration, the conservative preflop maximum is

$$2(169) + 150 + 1 + 42 + 5 = 536 \text{ tokens},$$

where 150 accounts for seat, scalar, event, and half-blind-tick features; the remaining terms are a global token, public history, and five configured action tokens. The corresponding current full-row postflop maximum is

$$2(1,326) + 150 + 6 + 42 + 5 = 2,855 \text{ tokens}.$$

On a flop, only $\binom{49}{2} = 1,176$ concrete hands avoid the three board cards. Thus 150 of the 1,326 rows are logically impossible for each example. The model has compatibility maps and compact gather/scatter machinery, but the conservative queued call currently retains the full padded width unless a fixed compact size is supplied. This is an excellent candidate for a measured optimization: it offers a clear $1,326/1,176 \approx 1.13$ row ratio, but only a trace can show whether removing those rows materially reduces end-to-end time.

These calculations illustrate three layers of "shape" that must not be confused:

- cumulative trajectory/topology capacity;
- retained storage width for one public state; and
- physical model sequence length after tokenization and padding.

Changing any one may leave the others unchanged.

16.3 The mode space

The core generates twelve mode contracts from

```
{self-play, best response}
× {sequential, branched fixed, branched KKT}
× {sampled, Rao--Blackwell}
```

Normal sequential training samples a concrete private outcome. Sequential RB training integrates additional hidden-state uncertainty through completion and mass/moment targets. Branched modes deliberately expand selected decisions. These variants share storage, rollout, target, and optimizer authorities while changing declared fields, multiplicity, estimator logic, and presentation.

16.4 Sampled versus Rao--Blackwell values: a toy example

Suppose a hidden opponent state is A with probability 0.6 and B with probability 0.4. For a fixed public decision, imagine the terminal contribution is $+1$ in state A and -1 in state B. A Normal sampled target draws one state. Its expectation is

$$0.6(1) + 0.4(-1) = 0.2,$$

but one sample is always $+1$ or -1, with variance $1 - 0.2^2 = 0.96$. An exact conditional expectation would publish 0.2 directly and remove that hidden-state sampling variance. This is the intuition behind Rao--Blackwellization. More generally,

$$\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(\mathbb{E}[X \mid Y]),$$

and replacing X by $\mathbb{E}[X \mid Y]$ removes the first nonnegative term.

The real project cannot enumerate every relevant completion for free. Its RB path constructs posterior/profile samples and mass/moment targets, then asks the evaluator-owned completion operation for the relevant hand-rank work. The selected H100 baseline used 1,024 completion samples. RB therefore trades more workspace and evaluator work for a potentially lower-variance target; changing that sample count is a scientific estimator change, not merely a batch-size tweak.

16.5 One decision node through the system

For a concrete mental execution, follow one postflop public node:

1. The fixed state owner holds stacks, commitments, board, statuses, reaches, and packed history on CUDA.
2. The scheduler places the row in a semantic family such as `post_2`, so rows with a compatible model contract can be executed together.
3. The model reconstructs public features and compatibility masks, evaluates attention over the private-hand representation, and emits value plus factorized fold/check-call/raise information for all 1,326 output rows.
4. A logical RNG address selects the action required by the estimator without making randomness depend on queue order.
5. The transition writes the child state into an unpublished reservation. Only after every field succeeds does the transaction publish the new tail.
6. A terminal child becomes a leaf. Sampled mode evaluates its sampled hidden state; RB mode also constructs completion work over posterior uncertainty.
7. The CPU-owned compact topology records structural relationships, while numerical leaf and layer data remain accelerator-resident.
8. Reverse-layer CUDA propagation produces decision targets. The learner then samples aligned public rows, gathers the relevant private targets, performs forward/loss/backward, and commits one dense FP32 Adam transition.

Thousands of such nodes share each physical call, and phases have different pack limits. The example is conceptually sequential, but the implementation's purpose is to batch independent rows at every honest axis while preserving the same transaction and estimator semantics.

16.6 Why this is a difficult GPU workload

Several tensions are present at once:

- trajectories are irregular, while GPUs prefer regular batches;
- public states are cheap relative to hidden-hand and completion domains;
- preflop and postflop use materially different model/input shapes;
- the simulator generates the learner's data, so production and consumption rates must be balanced;
- correctness requires schedule-independent randomness and transactional publication;
- RB improves an estimator but introduces large sample/workspace dimensions;
- independent replicas can expose parallelism but replicate persistent state; and
- the neural model is small enough that a large H100 can be launch- or shape-limited rather than peak-compute-limited.

The project is therefore a systems pipeline with neural kernels inside it, not merely "a transformer on a GPU."

17. Where the high-ROI ideas appear in the implementation

This section describes the architectural intent visible at the current repository revision. Exact class and field names can evolve; the systems ideas are more durable.

17.1 One canonical config becomes one resolved execution manifest

The scientific config names game, mode, model, objective, optimizer, schedule, and seed. Hardware execution config names the explicit device, precision, attention backend, reserve, and physical microbatch limits. A capacity compiler derives partition rows, topology and leaf bounds, token bounds, workspaces, and phase envelopes from that canonical input and the observed CUDA memory.

This addresses a subtle but high-value systems problem: duplicated derived configuration is executable disagreement. If YAML says one input width while the model constructs another, or one generator computes folded capacity from a pack while runtime retains it for a phase, the GPU eventually reveals the disagreement as an overflow or dtype/shape error. Derive facts once and save the resolved manifest beside the run.

17.2 Fixed SoA slabs and monotone publication

`SoASchema`, `MonotoneSlab`, `ExactBlock`, and generated owners provide fixed, aligned, accelerator-resident storage. Typed fields are views over byte arenas. One lane reserves, fills, and publishes monotone prefixes; capacity does not grow at runtime.

This combines performance and correctness:

- SoA supports coalesced field-wise kernels;
- fixed addresses reduce allocation churn and suit asynchronous execution;
- published tails are explicit producer/consumer boundaries;
- high-water marks make memory use auditable;
- whole failed attempts can be discarded; and
- an overflow fails clearly instead of paging state through CPU memory.

The tradeoff is that geometry proofs become first-class software. The history shows why that is worthwhile: wrong capacity is caught as a local invariant rather than becoming silent corruption.

17.3 Separate cumulative owners from bounded workspaces

The current capacity code distinguishes persistent/cumulative storage from bounded rollout, chance, completion, target, and learner packs. For example, `rollout_rows`, `completion_rows`, and `learner_rows` are independent physical execution limits even when the scientific learner batch remains 256.

This is exactly the logical/physical split from the generic section. It permits a large statistical batch or root count while scheduling each expensive phase with a different memory-optimal pack. It also explains why "batch 256" does not prove every internal model call contains 256 rows.

17.4 Explicit phase lifetimes

Rollout-only owners are released after sealing; leaves survive until completion; layer work survives until target publication; learner heads, targets, and indices survive through backward. Completion is intentionally serialized after rollout in the documented design so that evaluator workspace can reuse memory rather than compete with peak rollout state.

This sacrifices possible overlap in exchange for a smaller and more provable memory envelope. On an 80 GB H100, capacity often buys more useful root-level parallelism than speculative overlap would buy latency. The right decision is phase-specific and measured, but the current serialization has a coherent systems rationale.

17.5 Device-native rollout transactions and streams

`RolloutTransaction` is the common model/sample/transition/commit path. The implementation uses CUDA streams and events to express readiness and atomic publication. Logical RNG addresses separate random meaning from queue order. Failure metadata is device-resident until an explicit inspection boundary.

This avoids a common anti-pattern in simulators: move state to CPU after every step, branch in Python, then upload the next batch. The CPU still owns small scheduling and topology responsibilities, but numerical rollout state remains accelerator-native.

17.6 A compiled layered-target kernel

The rollout produces a compact topology. Reverse-layer target propagation is implemented by a compiled CUDA executor that launches one kernel per nonempty layer without a Python loop over individual nodes/actions or an inter-layer host synchronization. One thread block owns one parent and reduces children in logical order. Two work arenas alternate across layers, and short branches can remain in their producing slot until their last consumer rather than being copied.

This is a model high-ROI custom kernel target: stable semantics, repeated bounded reductions, clear ownership, and a host-loop/materialization cost that generic tensor expressions handled poorly. The phase-closure audit recorded a matched old target-propagation path at 14.670 seconds versus a 0.00576-second CUDA interval after the redesign. That historical comparison should not be generalized to the full application, but it illustrates the payoff of moving the right structural loop.

17.7 Optimized attention with an explicit backend

Poker's preflop and postflop models use reach-biased multi-head attention. The runtime accepts explicit `sdpa` or `chunked_dense` strategies rather than an implicit "auto" application-level fallback. PyTorch SDPA is the high-ROI route to optimized attention kernels; the chunked path provides a bounded alternative when its contract is selected.

There is an important evidence boundary: an SDPA configuration proves that the operator was requested, not that every real shape used Flash attention. The August H100 run configured and exercised BF16 SDPA but did not capture a formal kernel-dispatch proof. Future profiling should close that gap before attention performance conclusions are made.

17.8 The evaluator boundary

The sibling `poker-gpu-evaluator` contract owns showdown evaluation and the completion operations that produce `LayerWork` involving hand ranks. This repository owns public game state, policy, likelihood reaches, posterior/profile sampling, and folded summaries. There is no CPU poker-math fallback.

That is both a modularity and performance decision. One bound evaluator per deck geometry can retain/reuse its CUDA workspace. A strict interface prevents an innocent fallback from staging enormous hidden-hand computation through host memory.

17.9 Dense FP32 optimizer state and transactional learning

Model execution may use autocast BF16, while dense parameters/gradients, optimizer controls, and moments retain FP32 semantics. The learner transaction protects model, optimizer, RNG, replica version, and run state so a late lane or rank failure does not partially publish an iteration.

This costs memory. The current design chooses debuggability and exact rollback over a smaller but fragile state machine. Optimization should focus on batching finite checks, clipping, and Adam operations without changing per-lane norms or reductions.

17.10 A concrete replica-memory calculation

The production HU configuration contains 4,531,771 preflop and 6,602,467 postflop parameters, or 11,134,238 total. One FP32 vector over every parameter therefore occupies

$$4(11,134,238) = 44,536,952 \text{ bytes} \approx 42.47 \text{ MiB}.$$

The current learner/reference audit expands that vector into the following persistent ownership per independent replica:

Persistent state	Approximate bytes
Learned parameters	44,536,952
Dense gradients	44,536,952
Two Adam moments	89,073,904
Parameter and two-moment rollback snapshots	133,610,856
Frozen reference parameters	44,536,952
Total before small buffers	**356,295,616 (0.332 GiB)**

BF16 autocast does not halve these quantities because their declared owner is FP32. At $K = 32$, persistent learner/reference state alone is already about $32(0.332) = 10.6$ GiB before activations, rollout owners, completion workspaces, targets, reference caches, allocator reserve, or safety margin.

This is why replica count and physical learner batch must be planned jointly. The audit's empirical learner-only increment was approximately

$$M_{\text{active}}(K, B) \approx K\,(0.332 + 0.00360B) \text{ GiB},$$

where B is the simultaneously active physical minibatch per replica. For example, $K = 32, B = 64$ estimates about 18.0 GiB for learner-side persistent state and activations, but that number is not a whole-session admission proof. It was fit from a narrow set of matched H100 runs and omits other phases. The correct workflow is to use it to choose a candidate, compile a full lifetime envelope, then validate the warmed steady-state high-water mark.

18. H100 versus RTX 2080 Ti: not the same GPU at different speed

The local server uses RTX 2080 Ti hardware for development and CUDA regression; the paid/cluster target is an NVIDIA H100 80 GB HBM3. The relevant H100 run identified an SXM-class device with compute capability 9.0, 85,017,493,504 bytes visible to PyTorch, BF16 support, and a 700 W power cap. The comparison below uses NVIDIA reference/Founders Edition RTX 2080 Ti figures and H100 SXM specifications; installed board clocks and software can differ.

Property	RTX 2080 Ti	H100 SXM 80 GB	Consequence here
Architecture	Turing TU102	Hopper GH100	Different kernels and tuning rules
Compute capability	7.5 (SM75)	9.0 (SM90)	Extensions need correct targets/caches
SM count	68	132 enabled in SXM H100	More parallel capacity to feed
Device memory	11 GB GDDR6	80 GB HBM3	About 7.3× nominal capacity
Peak memory bandwidth	616 GB/s	3.35 TB/s	About 5.4× specification bandwidth
Peak non-Tensor FP32	14.2 TFLOP/s FE	67 TFLOP/s	About 4.7× specification peak
Low-precision Tensor math	FP16; 113.8 Tensor TFLOP/s FE	BF16/FP16; 989 dense or 1,979 sparse Tensor TFLOP/s	H100 adds BF16 and a much larger matrix ceiling
BF16 Tensor Core support	No	Yes	Local BF16 production path cannot run
Nominal board/TDP	260 W FE	up to 700 W	Different thermal/power environment
Memory technology	GDDR6	HBM3	Capacity and bandwidth change geometry choices

H100's published low-precision figures require careful reading: NVIDIA lists 1,979 BF16/FP16 Tensor TFLOP/s with sparsity for H100 SXM, implying 989 TFLOP/s for the corresponding dense specification. Neither figure predicts this project's throughput. Poker routing, small GEMMs, attention shapes, launch gaps, completion, targets, and optimizer work all sit outside a single dense matrix peak.

18.1 What the 2080 Ti is good for

The local Turing GPU is valuable for:

- CUDA residency and no-CPU-fallback gates;
- FP32 control-flow, storage, target, evaluator, and learner tests;
- memory-safety checks at small geometry;
- deterministic sampling and transaction/rollback tests;
- SM75 extension compilation; and
- cheap iteration on profiler annotations and benchmark harnesses.

Its smaller capacity can even expose memory waste earlier. An implementation that only works because an H100 has 80 GB is not necessarily well designed.

18.2 What it cannot establish

The RTX 2080 Ti cannot establish:

- BF16 forward/backward correctness or performance;

- SM90-native extension compatibility;
- Hopper-specific Flash/SDPA or grouped-GEMM dispatch;
- H100 Tensor Core efficiency;
- the root/replica geometry admitted by 80 GB;
- HBM3 behavior;
- Hopper concurrency or launch/compute balance; or
- the exact framework compatibility surface on the rental image.

This is why the local pilot deliberately used FP32, tiny roots, batch 8, one gradient step, and a small RB completion count. Those are smoke semantics, not H100 throughput estimates.

18.3 Newer hardware can expose more bugs

Moving to H100 activates paths that local testing never executes:

- BF16 storage and autocast;
- much larger cumulative geometries;
- SM90 compilation and kernel selection;
- library implementations chosen only for newer hardware;
- enough concurrency to expose race/order defects; and
- longer or more expensive arms where missing progress telemetry matters.

It also makes host overhead more visible. If a fixed CPU launch sequence takes the same time on both machines while each GPU kernel becomes faster, the H100 spends a larger fraction of time waiting. Peak hardware magnifies systems inefficiency.

19. What the H100 debugging history actually taught us

The repository contains two related H100 episodes: an initial late-July pilot and a broader August 2 RunPod selection. The following is a compact chronology, not a claim that every historical branch equals the current checkout.

19.1 Baseline rewrite and initial H100 pilot

By July 28, the queued accelerator-only engine, fixed storage, transactional rollout, evaluator integration, and layered targets had replaced earlier execution surfaces. H100 pilot tooling and Normal/RB configs followed on July 29. A RunPod pilot proved that one-replica HU 32 bb Normal and RB could execute in BF16 with 512 roots, logical batch 256, and 116 learner steps. The audit reported about 42.13 Normal and 41.12 RB player nodes/s, with peak reserved memory no greater than 16.38 GiB for those successful arms.

That success did not mean the launch path was clean. It produced four concrete blockers/lessons.

19.2 Failure 1: FP32 model input routed into BF16 storage

Symptom. A BF16 smoke failed when strict `index_copy_` attempted to write an FP32 `model_input` into a BF16 family arena.

Cause. Upstream model-input construction intentionally produced FP32, while the H100 execution manifest declared BF16 model storage. The route boundary had not encoded the allowed conversion.

Repair. Cast only floating-point `model_input` at that declared boundary. Keep every other dtype mismatch fatal, especially integer addresses, owners, masks, and metadata.

General lesson. Mixed precision needs typed interface boundaries. "Make PyTorch cast whatever is convenient" is unsafe; "never cast" is also wrong when storage precision is an explicit design choice.

The validated RunPod repair existed first as `2ebfae7` and was ported into the main line by `dcb120a` on July 31 with focused tests.

19.3 Failure 2: a cumulative folded-range owner sized as one pack

Symptom. With 512 roots and a scheduler pack of 84, the folded-range slab overflowed deterministically when the fifth pack began at logical root 336.

Cause. Configuration used roughly `min(family_pack, roots)` rows for data that survived across the complete rollout. That formula described simultaneous producer work, not cumulative retained ownership.

Repair. For the pilot, size the per-partition folded capacity by the full-root lifetime; the generalized bound accounts for every possible folder. Later manifest compilation derived such capacities centrally rather than requiring duplicated YAML arithmetic.

General lesson. Capacity is a theorem about lifetime. The error was not "H100 needs more memory"; it was "the formula bounded the wrong object." Adding a generous constant would have hidden the misunderstanding.

The validated pilot repair was `d9db16c`; the main-line port again landed in `dcb120a`.

19.4 Failure 3: K replicas performed the learner twice

Symptom. Normal K=1 took 39.69 seconds while K=2 took 158.82 seconds; RB K=1 took 41.52 seconds while K=2 again took 158.82 seconds. Doubling work caused approximately 4× time, so aggregate throughput fell to about half.

Cause. The replica/distributed path first ran a complete validate-only forward/loss/backward for every lane, cleared it, and then repeated the same work for committed updates. Lanes were also processed serially. K=4 reached high sampled GPU utilization but timed out; it was doing abundant redundant work, not demonstrating efficient parallel learning.

Repair. Commit `9229734` removed the duplicated learner preflight and introduced one transactional learner pass per lane with rollback/failure agreement over model, optimizer, RNG, replica version, and run state.

General lesson. Utilization and correctness machinery must be included in the work ledger. A GPU at 95% utilization can be productively wrong, or merely busy computing the same gradients twice.

19.5 Failure 4: long jobs produced insufficient evidence

Symptom. Timed-out arms could leave empty logs and no partial benchmark report. Aggregate timing did not explain whether rollout, completion, targets, learning, or synchronization was responsible.

Repair. The benchmark added atomic progress reports at warmup/repeat boundaries. Later H100 work used detached launches, per-arm exit files, metrics, GPU monitoring, and harvested artifacts.

General lesson. Observability is production functionality. A benchmark that cannot explain or resume a failure wastes both hardware rental time and the opportunity to improve the model.

19.6 Correctness found during performance preparation

The H100 preparation also found a statistical issue. An automatic cost-aware sequential allocation favored cheap preflop rows (roughly 239/17 or 238/18 pre/post in representative retained data) but weighted the learner as though the allocation were the target distribution. This changed the effective decision objective.

Commit `ef22ee2` changed default fixed-step sequential training to sample with replacement from the pooled retained decision extent, grouping by family only for physical model execution. Explicit cost-stratified allocation remains possible with normalized target-mass weights, and physical microbatch slices retain exact logical-batch weight.

General lesson. Performance-oriented stratification is importance sampling whether or not the code calls it that. Proposal probabilities and target mass must be reconciled mathematically before comparing throughput.

19.7 August 2 environment compatibility surprises

The larger RunPod experiment started from `ed4a73e` on an H100 80 GB HBM3 with driver 580.126.09, PyTorch 2.4.1+cu124, CUDA runtime/toolkit 12.4, SM90, BF16, and MIG disabled. Preflight found two more operational defects:

1. Collecting framework and evaluator tests in one pytest process caused top-level `tests` package shadowing. Running the suites in separate processes fixed the invocation without source changes.
2. PyTorch 2.4.1 CUDA did not support stable `argsort` directly on boolean keys. Casting the sort key to `int32` preserved `False=0`, `True=1`, stable row order, and scientific semantics. This became experiment commit `3e563319`.

A reporting bug used nonexistent `IterationResult.steps` instead of `optimizer_steps`, so the explicit gradient telemetry block was absent despite successful finite updates. The experiment fixed that in `8e305ca`. Neither experiment commit was present at the documented starting main-line revision; the handoff retained a bundle and patches rather than claiming they had been pushed.

General lesson. Framework version is part of the executable contract. A valid operation on one CUDA/PyTorch stack may have a dtype-specific hole on another. Preflight should exercise real dtypes and operators before launching the expensive matrix.

20. What the August H100 experiment measured

The August 2 experiment was explicitly a fresh-initialization execution benchmark. It did not compare trained policy quality, exploitability, convergence, or estimator equivalence. Every launched experimental arm exited successfully; the final Normal and RB reliability runs each completed three iterations, 116 finite-gradient optimizer commits per iteration, checkpoints, and clean shutdown.

The selected HU 32 bb scientific/execution baseline was:

Setting	Selected value
replicas K	1
roots	4,096
rollout/completion/learner packs	256 / 512 / 256
logical learner batch	256
learner steps	116
precision	BF16 autocast with FP32 training state
attention	SDPA
reference cache	auto
RB completion samples	1,024

The pilot model used $d_{\text{model}} = 128$, four heads, four preflop layers, six postflop layers, and two raise components.

20.1 The decisive result was root parallelism, not serial replicas

Fair cache-off K comparisons at 512 roots retained K=1:

Mode	K=1	K=2	Aggregate K=2 gain	Decision
Normal	13.220 nodes/s	13.345 nodes/s	0.95%	keep K=1
RB	12.837 nodes/s	13.235 nodes/s	3.10%	keep K=1

Those gains were below the predeclared 10% threshold. Short K=4 probes at 64 roots per replica passed stability/memory gates but were not fair throughput comparisons; K=8 was not run.

Increasing roots was dramatically more valuable:

Mode	512 roots	2,048 roots	4,096 roots
Normal	23.662 nodes/s	83.442 nodes/s	158.121 nodes/s
RB	23.069 nodes/s	85.160 nodes/s	153.293 nodes/s

The arithmetic makes the Pareto result concrete. Multiplying roots by eight from 512 to 4,096 multiplied Normal throughput by about 6.68 and RB throughput by about 6.64. That is roughly 84% and 83% of ideal linear scaling over this interval. In contrast, doubling serial replicas improved aggregate throughput by only 0.95% and 3.10%. The root sweep did not make each root cheaper in every abstract sense; it gave the H100 enough independent rows to amortize fixed costs and occupy more of the machine.

This does not prove replica batching is a bad idea. The tested K path remained largely serial at the learner level, so it mostly replicated work and state. It proves that, for the implementation actually measured, using H100 memory to expose more root-level work was much higher ROI than adding serial replicas.

20.2 Steady-state memory was much higher than one-iteration memory

The selected single-iteration arms reported first-iteration peak reserved memory of 26.498 GiB Normal and 34.736 GiB RB. Three-iteration reliability runs, which matured the automatic reference cache, reached conservative cumulative peaks of 50.271 GiB and 58.619 GiB.

Normal cumulative peak plateaued by iteration 3. RB changed by only about 50.3 MB (0.08%) from iteration 2 to 3 while end-reserved memory decreased, consistent with stable workload/allocator variation rather than sustained growth.

This is a textbook memory-lifetime lesson: a one-step peak was not the session peak because a later persistent cache changed the live set.

The measured H100 exposed about 79.18 GiB to PyTorch. The 58.619 GiB RB peak therefore left about 20.6 GiB between the recorded high-water mark and visible capacity. That difference is not an invitation to allocate 20.6 GiB: the design still reserves 10 GiB for safety, and future shapes, library workspaces, fragmentation, compiled graphs, or cache maturation can consume part of the rest. Admission must compare the full phase envelope with the configured reserve, not merely check that the previous run did not OOM.

20.3 Optional knobs did not justify scientific changes

Reducing learner steps from 116 to 58 shortened wall time and increased rollout nodes/s, but reduced learner presentations/s. That is a schedule change, so the experiment retained 116 without learning-quality evidence.

At logical learner batch 256, one replica performs

$$116 \cdot 256 = 29,696$$

learner presentations per iteration, or 89,088 across the three-iteration reliability run. A 58-step arm performs exactly half that learning work. Its shorter wall time is useful capacity-planning evidence, but calling it an executor speedup would compare different algorithms.

Reducing RB completion samples from 1,024 to 512 improved measured nodes/s by only about 0.8%. Because sample count changes estimator quality and no variance or exploitability equivalence was established, the experiment retained 1,024.

20.4 The final measured baseline

Across the three-iteration steady-state confirmations, the handoff reported:

- Normal: 169.563 player nodes/s and 459.778 learner presentations/s at a 50.271 GiB measured steady-state peak;
- RB: 174.111 player nodes/s and 456.481 learner presentations/s at a 58.619 GiB measured steady-state peak.

These are execution baselines for that exact fresh-initialization environment, not general H100 claims. The formal Flash-attention dispatch proof remained open, and a PyTorch `vmap` batching-fallback warning appeared for efficient SDPA in targeted tests.

21. A Pareto-ranked experiment and optimization backlog

The following is a proposal, not a description of already implemented production behavior. Its order is chosen to maximize information and reusable value before specialized engineering.

Tier 0 — Reconcile and reproduce the exact target

1. **Reconcile experiment-only fixes.** Decide whether the boolean-sort and gradient-telemetry commits from the August handoff belong on the intended branch. Run the full CUDA gates after any port; do not treat a patch archive as current source.
2. **Reproduce the selected 4,096-root configs on the target Quest H100.** Record exact SKU, driver, toolkit, PyTorch/cuDNN, evaluator revision, native SM90 extension identity, clocks, MIG state, and contention.
3. **Prove SDPA dispatch.** Force or inspect Flash forward and backward on real preflop/postflop shapes. If a shape falls back, record why and its runtime coverage before changing the model.
4. **Validate checkpoint/resume and trained initialization.** The August run was fresh-initialization execution evidence. A scientific run needs preserved model provenance and resume identity.
5. **Add phase-complete progress.** Warmup/repeat reports are useful; longer production runs also benefit from phase and learner-step heartbeats without introducing hot-path synchronization.

These steps are high ROI because every later speed comparison depends on them.

Tier 1 — Improve the measured K=1 critical path

1. Capture an unprofiled phase breakdown and an Nsight Systems trace for a fixed learner replay and one full iteration.
2. Separate learner core time (staged inputs through optimizer) from gathering, rollout, completion, targets, reference cache, and checkpoint work.
3. Record every dominant GEMM/attention shape and exact physical/useful token count. Compare kernels with isolated same-shape ceilings.
4. Sweep a narrow neighborhood around 4,096 roots and the 256/512/256 packs, subject to the 10 GiB reserve and steady-state cache peak. The earlier sweep found a steeply rising curve; it did not necessarily locate the plateau.
5. Compile stable learner regions and compare eager, compiled, and—only for a bounded static shape set—CUDA Graph replay. Track compilation count, graph pool memory, and numerical drift.
6. Vectorize finite checks, norm/clipping, gradient clearing, and Adam operations where profiling shows per-parameter or per-lane launch amplification.

The retention criterion should be end-to-end or full-replay gain, not an operator microbenchmark alone.

Tier 2 — Make replicas visible to the GPU

The existing K=2 result does not answer whether a truly batched replica learner could saturate H100 better. Build one immutable replay artifact so candidates receive identical per-lane inputs, targets, sampling order, initial parameters, moments, and RNG.

Compare these implementations:

1. **Serial oracle:** current one-lane-at-a-time learner.
2. **CUDA stream waves:** unchanged per-lane model on multiple streams; cheapest test of whether independent kernels leave complementary resources idle.
3. **Functional `vmap` plus outer backward:** one canonical model with stacked state; reject batching fallbacks and loss of Flash backward.
4. **Explicit replica-native BMM:** `[K,...]` weights, batched linear primitives, and replica folded into the SDPA batch.
5. **Grouped GEMM for tails:** only if measured padding waste is material and the target software/SM90 kernel is stable.

The scheduler should be **step-major**, not lane-major:

```
for logical step s:
    prepare the next batch for every active lane
    execute the lane cohort
    validate all lanes/ranks
    apply independent per-lane transitions
```

Within each lane, step s must commit before step $s+1$. Across lanes, work can be concurrent. Loss normalization, clipping, moments, RNG, and rollback remain lane-local. A summed cohort loss is acceptable for one outer backward only if it is a sum of independent lane losses and gradients land in disjoint stacked parameter rows; averaging across K would alter each transition.

Suggested gates before production retention:

- K=1 regression no worse than 3%;
- K=2 aggregate learner gain at least 25%;
- K=4 gain at least 75%;
- at least 10% over the next-best correct new executor;
- exact logical samples and per-lane isolation;
- FP32 one-step oracle comparison and predeclared BF16 error bounds;
- complete 116-step drift, checkpoint, and failure-injection gates;
- Flash forward/backward retained; and
- no memory growth with the configured reserve.

These are proposed decision thresholds, not promises that the hardware will meet them.

Tier 3 — Reduce model-side wasted work without mixing the experiment

Potentially valuable ideas include:

- compact board-impossible postflop hand rows before expensive attention;
- evaluate whether the full all-hands readout is required for every Normal presentation;
- reduce avoidable pre/post padding through measured buckets;
- remove redundant incidence/folded features in a heads-up-specific model; and
- revisit long-sequence attention structure.

Some of these alter architecture or the domain of outputs. They must be tested as separate scientific factors with learning-quality evidence. A faster heads-up-lite model cannot establish that an executor is faster, and it cannot silently replace the common Normal/RB comparison model.

Tier 4 — Custom kernels after profiling

Candidate custom work already suggested by design audits includes:

- fused posterior/profile sampling without a large dense attempt envelope;
- direct selected-action policy computation rather than materializing every legal-action probability;
- fused transition/folded-statistic construction;
- explicit replica linear primitives where library batching is poor; and
- persistent kernels only for a demonstrated dominant fixed shape.

For each candidate calculate Amdahl coverage from a real trace. Require a credible full-step upside before implementation and a material measured gain after integration. Keep one ordinary correctness oracle and delete losing production backends after the experiment; a permanent forest of switches is a systems cost.

22. A concrete experiment matrix

The safest way to spend scarce H100 time is a staged, resumable matrix.

Stage A: local RTX 2080 Ti gates

- Run FP32 one-step and 116-step replays for every candidate available on SM75.
- Test exact sampling, lane isolation, rollback, checkpoint/resume, and memory reconciliation.
- Compile extensions into an SM75-specific cache.
- Runtime-gate SM90-only grouped kernels rather than pretending the local test covered them.

Stage B: five-minute H100 preflight

- Confirm an exclusive non-MIG H100, identity, power/clocks, and clean memory.
- Compile/load every native extension from an SM90-specific cache.
- Run tiny BF16 Normal and RB forward/backward/completion.
- Verify real Flash attention forward/backward dispatch.
- Verify profiler permissions and every output/progress path.

Stage C: correctness and dispatch

- K=1 serial oracle for full preflop and postflop widths.
- K=2 one-step and complete-replay tests for each executor.
- Isolated forward, input-gradient, and weight-gradient benchmarks for dominant linear shapes.
- Explicit useful versus padded work accounting.

Stage D: broad unprofiled sweep

- Sweep admitted $K \in \{1, 2, 4, 8\}$ and physical learner cohorts around 16/32/64/128 rows where meaningful.
- Sweep fixed total cohort work KB to separate "more replicas" from "more model work."
- Include preflop, postflop, mixed learner replay, and complete iteration.
- Record steady-state timing and memory after caches/graphs mature.

Stage E: profile only finalists

- Capture bounded Nsight Systems traces with phase annotations.
- Profile representative GEMM/BMM, Flash forward/backward, incidence, reduction, and optimizer kernels with Nsight Compute.
- Compare achieved performance to isolated same-shape ceilings and the relevant rooflines.

Stage F: reliability and scientific handoff

- Alternate at least several complete Normal/RB repeats.
- Run multiple iterations to expose persistent caches and allocator growth.
- Verify checkpoints, resume, exact work ledgers, finite metrics, and clean teardown.
- Report speed, memory, correctness, and scientific limitations together.

Stop a candidate immediately when it changes logical work, crosses a numerical gate, couples replica state, loses required attention dispatch, exceeds memory, or cannot explain its speedup. A failed `vmap` experiment does not disprove explicit BMM; a failed BMM experiment does not disprove stream waves. It only answers the specific implementation tested.

23. Closing perspective

The main lesson of the project is not that H100 is "faster" than RTX 2080 Ti. It is that a larger accelerator changes the active constraints.

The 2080 Ti is an excellent correctness laboratory for common CUDA behavior, but it cannot execute the H100 BF16 and SM90 contract. The H100's extra memory made 4,096-root runs possible and those roots delivered far more aggregate value than serial replica scaling. At the same time, H100-specific execution exposed a missing precision cast, a lifetime-bound capacity error, framework operator compatibility, insufficient progress telemetry, and redundant learner work. None of those problems would be solved by quoting Tensor TFLOPS.

The project's highest-return systems choices are correspondingly plain:

- define and count correct statistical work;
- derive one immutable capacity plan from one config;
- keep production state accelerator-resident in typed, lifetime-aware storage;
- use device transactions and logical RNG so scheduling can change safely;
- call optimized attention/evaluator primitives and prove their dispatch;
- increase the cheapest honest parallel axis until throughput plateaus;
- measure steady state, including caches and rollback state; and
- specialize kernels only where a real trace and Amdahl's law justify them.

That is the Pareto view of GPU engineering: understand the bottleneck at the scale of the whole system, make one falsifiable change, retain it only when the useful workload becomes materially better, and let evidence—not the novelty of the technique—choose the design.

Glossary

Arithmetic intensity: Operations performed per byte moved from a chosen memory level.

Autocast: Framework policy that selects lower precision for eligible operations while leaving other operations in higher precision.

BF16 / bfloat16: A 16-bit floating format with an FP32-like exponent range and reduced significand precision.

Block / CTA: A cooperative group of CUDA threads scheduled together on one SM. CTA means cooperative thread array and is commonly used as a synonym for thread block.

CUDA Graph: An instantiated device-operation dependency graph designed for lower-overhead replay of a stable schedule.

Device: The GPU in CUDA terminology; the CPU is the host.

GEMM / BMM: General matrix multiplication / batched matrix multiplication.

HBM: High-bandwidth memory, the device DRAM technology used by the H100 SXM here.

Kernel: A function launched for parallel execution on the GPU.

Logical batch: The examples defining one statistical update, independent of physical slicing.

Microbatch / pack: A bounded physical subset executed concurrently to control memory or shape.

Occupancy: Resident warps relative to an SM's architectural maximum, constrained by block resources. It is a diagnostic, not the optimization objective.

Rao--Blackwellization (RB): Replacing some sampled uncertainty with a conditional expectation, typically reducing estimator variance at additional compute/memory cost.

Replica: One independent model, optimizer, RNG, and run state. Replica batching must not share weights or reduce gradients across replicas.

SDPA: Scaled dot-product attention; a high-level operator that may dispatch to Flash, memory-efficient, or math implementations depending on its inputs and stack.

SIMT / warp: CUDA's single-instruction, multiple-thread execution model; a warp contains 32 threads.

SM / SM75 / SM90: Streaming multiprocessor; SM75 and SM90 informally denote compute capabilities 7.5 (Turing) and 9.0 (Hopper).

SoA: Structure of arrays, a layout placing each field in its own dense array.

Tensor Core: Specialized hardware for supported matrix multiply-accumulate operations.

References and repository evidence

Generic primary references:

- NVIDIA CUDA Programming Guide
- NVIDIA CUDA C++ Best Practices Guide
- NVIDIA Hopper Tuning Guide
- NVIDIA Turing Tuning Guide
- NVIDIA Hopper architecture in depth
- NVIDIA Turing architecture whitepaper
- NVIDIA H100 specifications
- NVIDIA GeForce RTX 2080 Ti specifications
- PyTorch scaled dot-product attention
- PyTorch `torch.compile`

Project evidence used for the case study:

- `README.md`
- `docs/queued/core_pipeline.md`
- `docs/queued/storage_and_lifetimes.md`
- `docs/queued/topology_and_targets.md`
- `docs/queued/h100_pilot.md`
- `docs/audit/queued_issues/quest_h100_launch_blockers_20260729.md`
- `docs/audit/h100_normal_rb_execution_changes_20260802.md`
- `runs/experiments/nlhe_hu_32bb_h100_pilot/run_state*.json`
- `runs/nlhe_hu_32bb_runpod_h100_20260802_handoff/{RESULTS,DECISIONS,AUDIT,HANDOFF}.md`
- Git history through `ed4a73e`, plus the two experiment-only handoff commits `3e563319` and `8e305ca` recorded in that bundle

The untracked `docs/audit/h100_idea_*.md` and `docs/audit/h100_plan.md` files were consulted as proposal material only. This chapter does not treat their suggested executors, thresholds, or model variants as implemented production features.